FLIGHT MANAGEMENT SYSTEM

UCS310- DATABASE MANAGEMENT SYSTEM

B.Tech (2nd Year)

Computer Science and Engineering

2025-26 EVEN Semester

Submitted by:

Roopneet Cheema (1024030194)
Karaninder Singh (1024030400)

Submitted to:

GAYATRI SAXENA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY
PATIALA – 147004

May 2026

Introduction

A Flight Management System (FMS) is a software application designed to manage and automate airline operations such as flight scheduling, aircraft assignment, crew management, passenger reservations, ticketing, and real-time flight status tracking. It acts as a centralized platform that integrates operational, administrative, and customer-related data to ensure smooth and efficient airline functioning. By handling large volumes of data accurately and in real time, a flight management system helps airlines improve operational efficiency, safety, and customer satisfaction.

Motivation

The motivation for choosing a Flight Management System lies in its real-world relevance and complexity. The aviation industry deals with massive amounts of data, including flight details, passenger information, crew schedules, and maintenance records, all of which must be processed reliably and quickly. Designing this system provides practical exposure to handling real-time data, concurrency, data integrity, and scalability challenges. Additionally, it reflects real-life enterprise-level applications, making it an ideal problem domain for understanding how databases support critical systems.

Importance of Using a Database Management System Instead of a File-Based System

Comparison between a Database Management System (DBMS) and a File-Based System for a Flight Management System:

Aspect	DBMS	File-Based System
Data Redundancy	Minimal	High
Data Consistency	Maintained	Not guaranteed
Concurrency	Supports multiple users	Poor multi-user support
Security	Strong access control	Weak security
Data Retrieval	Fast, query-based	Slow, manual
Backup & Recovery	Automatic	Manual/unreliable
Scalability	Highly scalable	Limited

Problem Statement

Definition of the Real-World Problem:

Airlines manage large volumes of data related to flights, aircraft, crew, passengers, and bookings. Handling this information manually or through disconnected systems leads to inefficiencies, delays, and errors. The problem is to design a Flight Management System that efficiently stores, manages, and retrieves airline operational data in a reliable and centralized manner.

Limitations of the Existing / Manual System:

- Data redundancy and inconsistency
- Time-consuming manual record keeping
- High chances of human errors
- Difficulty in handling concurrent users
- Poor data security and access control
- Limited scalability as data grows

Scope of Database Usage in Solving the Problem:

- Centralized storage of flight, passenger, and crew data
- Fast and accurate data retrieval using queries
- Ensuring data integrity through constraints
- Supporting multi-user access and real-time updates
- Secure handling of sensitive information
- Easy backup, recovery, and scalability

Objectives of the Project

- To design the database structure using E–R data modeling
- To convert the E–R model into relational tables
- To apply normalization techniques up to 3NF / BCNF
- To implement the database using SQL
- To use PL/SQL constructs such as procedures, functions, triggers, and cursors
- To ensure data consistency and integrity using constraints and transactions

Scope of the Project

Functional Boundaries of the System:

- Management of flight schedules and routes
- Passenger reservation and booking records
- Aircraft and crew information management
- Storage and retrieval of operational data
- Database-level validation and transaction handling

Types of Users Involved:

- **Admin:** Manages system data, users, and overall database control
- **Staff:** Handles flight schedules, crew assignments, and updates
- **User (Passenger):** Views flight details and booking information

Modules Covered:

- Flight information management module
- Passenger and booking management module
- Aircraft and crew management module
- User access and role management module
- Transaction and integrity control module

Proposed System Description

The proposed system uses a centralized relational database to store all flight-related data. Authorized users interact with the system through structured SQL operations, while constraints and transactions ensure reliable data storage and updates.

Key Features and Functionalities:

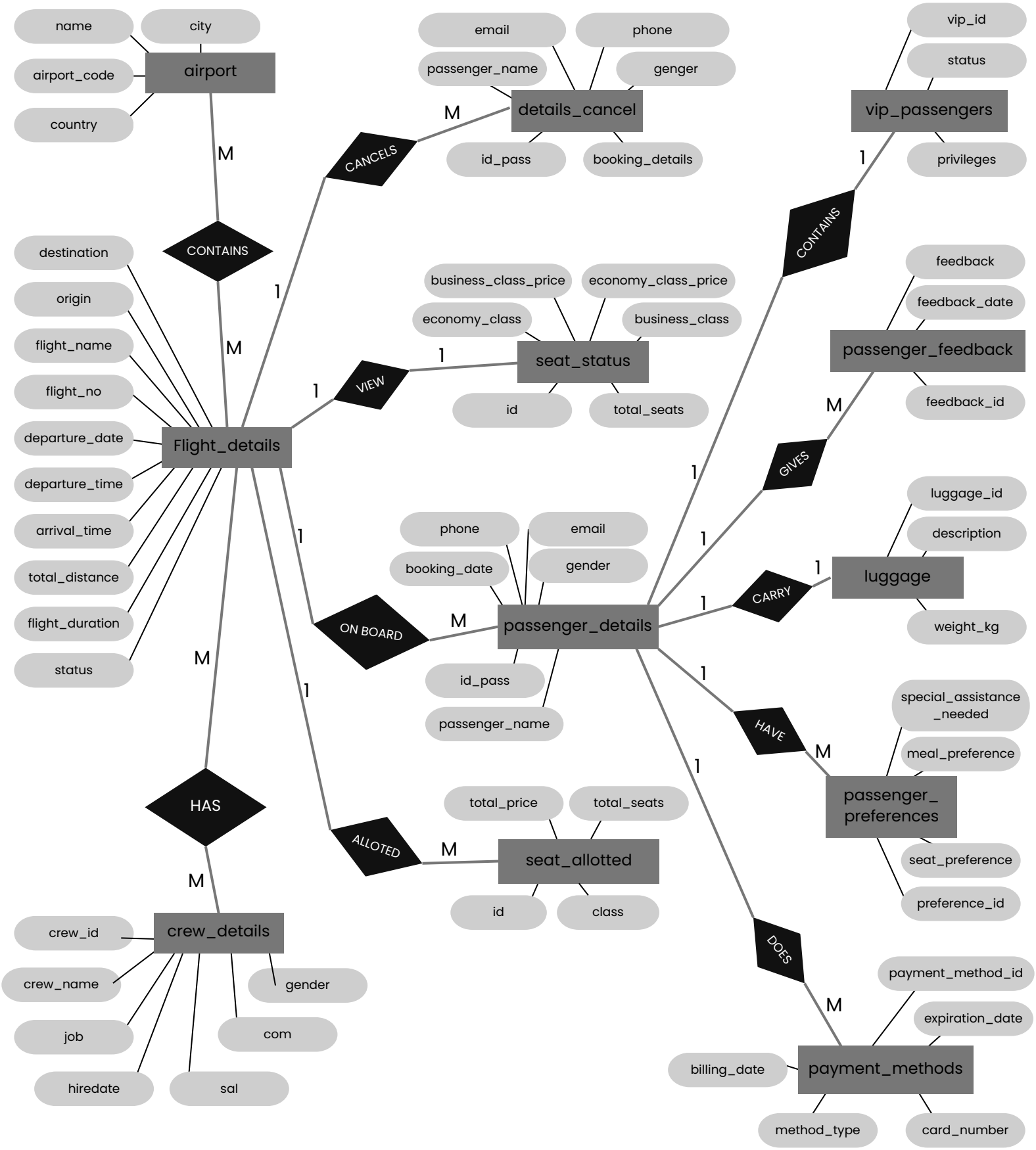
- Centralized and normalized database design
- Fast data retrieval using optimized SQL queries
- Role-based access control
- Automatic enforcement of data constraints
- Support for stored procedures, triggers, and functions

Improvements Over the Existing System:

- Efficiency: Reduces manual effort and processing time
- Consistency: Eliminates redundant and conflicting data
- Data Integrity: Enforced through keys, constraints, and transactions

Database Design

Entity–Relationship (ER) Diagram:



Er to Table

- **Flight_details** (flight_no, flight_name, origin, destination, departure_date, departure_name, arrival_date, arrival_time, total_distance, flight_duration, status)
- **Crew_details** (crew_id, crew_name, gender, job, hiredate, sal, comm)
- **Passenger_details** (passenger_name, email, phone_number, gender, booking_date)
- **Seat_status** (id, business_class, business_class_price, economy_class, economy_class_price, total_seats)
- **Seat_alloted** (flight_no, class, total_seats, total_price)
- **Details_cancel** (passenger_name, email, phone_number, gender, booking_date)
- **Airport**(airport_code,name,city,country)
- **Luggage**(luggage_id,idpassenger,weight_kg,description)
- **Passenger_preferences**(preference_id,passenger_id,seat_preference,meal_preference,speci al_assistance_needed)
- **Payment_methods**(payment_methods_id,passenger_id,method_type,card_number,expirati on_data,billing_address) 6
- **Passenger_feedback**(feedback_id,passenger_id,feedback,feedback_date)
- **Vip_passengers**(vip_id,flightno,id_passenger,status,privileges)

Relationship

• Flight_details has one to many relationship with Passenger_details.
• Flight_details has one to many relationship with Seat_status.
• Flight_details has one to one relationship with Seat_alloted. • Flight_details has one to many relationship with Details_cancel.
• Flight_details has many to many relationship with crew_details.
• Passenger_details has one to one relationship with vip_passengers.
• Passenger_details has one to one relationship with passengers_preference.
• Passenger_details has one to many relationship with passengers_feedback.
• Passenger_details has one to many relationship with luggage.
• Passenger_details has one to many relationship with payment_method.
• Airport has many to many relationship with Flight_details.

Normalization

FIRST NORMAL FORM:

The first normal form rule is that there should be no nesting or repeating groups in a table. Now an entity type that contains only one value for an attribute in an entity instance ensures the application of first normal form for the entity type. So, in a way any entity type with an entity identifier is by default in first normal form. Hence, all tables in our relational schema are in 1st normal form as none of the tables contain any composite attribute or multi-valued attribute or their combinations. As an example, let's look at the passenger_details table below:

Column Name	Data Type	Nullable
FLIGHT_NO	VARCHAR2(10 BYTE)	Y
PASSENGER_NAME	VARCHAR2(50 BYTE)	Y
ID_PASS	NUMBER	Y
EMAIL	VARCHAR2(50 BYTE)	Y
PHONE	VARCHAR2(100 BYTE)	Y
GENDER	VARCHAR2(10 BYTE)	Y
BOOKING_DATE	DATE	Y

In the Table passenger_details there is a multiple attribute named as Phone due to which this table is not in 1NF. To convert this table to 1NF Table is broken into 2 atomic tables which are shown below.
Table passenger containing all the information except passenger phone number:

Column Name	Data Type	Nullable
FLIGHT_NO	VARCHAR2(20 BYTE)	Y
PASSENGER_NAME	VARCHAR2(20 BYTE)	Y
ID_PASS	NUMBER(10,0)	N
EMAIL	VARCHAR2(50 BYTE)	Y
GENDER	VARCHAR2(10 BYTE)	Y
BOOKING_DATE	DATE	Y

Table passengerphone containing phone number of passengers:

Column Name	Data Type	Nullable
ID_PASS	NUMBER(10,0)	Y
PHONE	NUMBER(10,0)	Y

SECOND NORMAL FORM:

The second normal form rule is that the key attributes determine all non-key attributes. A violation of second normal form occurs when there is a composite key, and part of the key determines some non-key attributes. The second normal form deals with the situation when the entity identifier contains two or more attributes, and the non-key attribute depends on part of the entity identifier.
Let us look at table crew_details where:
(crew_id,crew_name)->{gender,job,hiredate,sal,comm} {crew_id}->{job}

Original table:

Column Name	Data Type	Nullable	Default	Primary Key
CREW_ID	VARCHAR2(10 BYTE)	Y		
CREW_NAME	VARCHAR2(50 BYTE)	Y		
GENDER	VARCHAR2(10 BYTE)	Y		
JOB	VARCHAR2(50 BYTE)	Y		
HIREDATE	DATE	Y		
SAL	NUMBER	Y		
COMM	NUMBER	Y		

The table already appears to be in 1NF since each cell contains a single value, and there are no repeating groups.In 2NF, we need to ensure that each non-prime attribute is fully functionally dependent on the entire primary key. In this case, the job attribute is dependent only on part of the primary key (crew_id).We need to split the table to remove this partial dependency. Now, the crew_details table contains information only about the crew members themselves, while the job_details table contains information about their respective roles. This separation ensures that each table contains data without any partial dependencies on the primary key.

The second normal form rule is that the key attributes determine all non-key attributes. A violation of second normal form occurs when there is a composite key, and part of the key determines some non-key attributes. The second normal form deals with the situation when the entity identifier contains two or more attributes, and the non-key attribute depends on part of the entity identifier.

Let us look at table crew_details where:
(crew_id,crew_name)->{gender,job,hiredate,sal,comm} {crew_id}->{job}

Original table:

CREW_ID	CREW_NAME	GENDER	JOB	HIREDATE	SAL	COMM
C002	Mary	Female	Flight Attendant	3/20/2023	35000	2000
C001	John	Male	Pilot	1/15/2023	80000	5000
C003	Alex	Male	Mechanic	5/10/2023	45000	3000

The table already appears to be in 1NF since each cell contains a single value, and there are no repeating groups.In 2NF, we need to ensure that each non-prime attribute is fully functionally dependent on the entire primary key. In this case, the job attribute is dependent only on part of the primary key (crew_id).We need to split the table to remove this partial dependency.

Now, the crew_details table contains information only about the crew members themselves, while the job_details table contains information about their respective roles. This separation ensures that each table contains data without any partial dependencies on the primary key.

Normalized tables:

CREW_DETAILS

Column Name	Data Type	Nullable
CREW_ID	VARCHAR2(10 BYTE)	N
CREW_NAME	VARCHAR2(50 BYTE)	Y
GENDER	VARCHAR2(10 BYTE)	Y
HIREDATE	DATE	Y
SAL	NUMBER	Y
COMM	NUMBER	Y

JOB_DETAILS

Column Name	Data Type	Nullable
CREW_ID	VARCHAR2(10 BYTE)	Y
JOB	VARCHAR2(50 BYTE)	Y

THIRD NORMAL FORM:

The third normal form rule is that the non-key attributes should be independent. This normal form is violated when there exists a dependency among non-key attributes in the form of a transitive dependency. Since a transitive dependency can be resolved by moving the dependency attributes to a new entity type with one-to-many relationship, we have used this property to solve the dependencies in our tables, making them in accordance with the 3rd normal form.
For 3NF following are the necessary conditions:
• If table is in 2NF.
• No non-key attribute is transitively dependent on the primary key.

Let us look at table flight_details:

Column Name	Data Type	Nullable	Default	Primary Key
FLIGHT_NO	VARCHAR2(10 BYTE)	N		1
FLIGHT_NAME	VARCHAR2(50 BYTE)	Y		
ORIGIN	VARCHAR2(50 BYTE)	Y		
DESTINATION	VARCHAR2(50 BYTE)	Y		
DEPARTURE_DATE	DATE	N		2
DEPARTURE_TIME	TIMESTAMP(6)	Y		
ARRIVAL_TIME	TIMESTAMP(6)	Y		
TOTAL_DISTANCE	NUMBER	Y		
FLIGHT_DURATION	NUMBER	Y		
STATUS	VARCHAR2(20 BYTE)	Y		

1. The table already appears to be in 2NF since it has a composite primary key (flight_no, departure_date), and each non-prime attribute is fully functionally dependent on the entire primary key.

2. Now, let's check for any transitive dependencies:
• total_distance and flight_duration both seem to be functionally dependent on flight_no alone, not on each other or any other non-prime attribute.
• flight_name, origin, destination, departure_time, arrival_time, and status also appear to be functionally dependent only on flight_no.
Since there are no transitive dependencies, the table is already in 3NF.

Similarly, all other tables are also in 3NF.

SQL Implementation

1. Flight_details:

```
CREATE TABLE flight_details (
   flight_no VARCHAR2(10) PRIMARY KEY,
   flight_name VARCHAR2(30) NOT NULL,
   origin VARCHAR2(30) NOT NULL,
   destination VARCHAR2(30) NOT NULL,
   departure_date DATE,
   departure_time TIMESTAMP,
   arrival_time TIMESTAMP,
   total_distance NUMBER,
   flight_duration NUMBER,
   status VARCHAR2(20)
);
```

```
2   select * from flight_details ;
```

Results | Explain | Describe | Saved SQL | History

FLIGHT_NO	FLIGHT_NAME	ORIGIN	DESTINATION	DEPARTURE_DATE	DEPARTURE_TIME	ARRIVAL_TIME	TOTAL_DISTANCE	FLIGHT_DURATION	STATUS
AI103	SpiceJet	Bangalore	Kolkata	4/26/2024	24-APR-26 11.28.07.061763 AM	24-APR-26 11.28.07.061763 AM	1500	2.8	Delayed
AI101	Air India	Delhi	Mumbai	4/24/2024	24-APR-26 11.27.53.907176 AM	24-APR-26 11.27.53.907176 AM	1400	2.5	On Time
AI102	Indigo	Amritsar	Delhi	4/25/2024	24-APR-26 11.28.02.115455 AM	24-APR-26 11.28.02.115455 AM	200	1	On Time

3 rows returned in 0.18 seconds Download

2. Crew_details:

```
CREATE TABLE crew_details (
   crew_id VARCHAR2(10) PRIMARY KEY,
   crew_name VARCHAR2(50),
   gender VARCHAR2(10),
   job VARCHAR2(50),
   hiredate DATE,
   sal NUMBER,
   comm NUMBER
);
```

```
1
2   select * from crew_details ;
```

Results | Explain | Describe | Saved SQL | History

CREW_ID	CREW_NAME	GENDER	JOB	HIREDATE	SAL	COMM
C001	John	Male	Pilot	5/11/2020	50000	10000
C002	Mary	Female	Co-Pilot	12/10/2021	30000	5000
C003	Gaurav	Male	Ticket Agent	1/1/2022	20000	5000

3 rows returned in 0.00 seconds Download

3.Seat_status:

```
CREATE TABLE seat_status (
   id NUMBER PRIMARY KEY,
   flight_no VARCHAR2(10),
   business_class NUMBER,
   economy_class NUMBER,
   business_class_price NUMBER,
   economy_class_price NUMBER,
   total_seats NUMBER,
   FOREIGN KEY (flight_no) REFERENCES flight_details(flight_no)
);
```

```
select * from seat_status;
```

Results | Explain | Describe | Saved SQL | History

ID	FLIGHT_NO	BUSINESS_CLASS	ECONOMY_CLASS	BUSINESS_CLASS_PRICE	ECONOMY_CLASS_PRICE	TOTAL_SEATS
2	AI102	30	60	6000	2500	90
1	AI101	20	60	10000	4000	80
3	AI103	40	80	5000	2000	120

3 rows returned in 0.03 seconds Download

4. Passenger_details:

```
CREATE TABLE passenger_details (
   id_pass NUMBER PRIMARY KEY,
   flight_no VARCHAR2(10),
   passenger_name VARCHAR2(50),
   email VARCHAR2(50),
   phone VARCHAR2(15),
   gender VARCHAR2(10),
   booking_date DATE,
   FOREIGN KEY (flight_no) REFERENCES flight_details(flight_no)
);
```

```
select * from  passenger_details;
```

Results | Explain | Describe | Saved SQL | History

ID_PASS	FLIGHT_NO	PASSENGER_NAME	EMAIL	PHONE	GENDER	BOOKING_DATE
101	AI101	Rahul	rahul@gmail.com	9876543210	Male	4/24/2026
103	AI103	Aman	aman@gmail.com	9988776655	Male	4/24/2026
102	AI102	Priya	priya@gmail.com	9123456780	Female	4/24/2026

3 rows returned in 0.05 seconds Download

5. Details_cancel:

```
CREATE TABLE details_cancel (
   cancel_id NUMBER PRIMARY KEY,
   passenger_id NUMBER,
   cancel_date DATE,
   FOREIGN KEY (passenger_id) REFERENCES passenger_details(id_pass)
);
```

```
INSERT INTO details_cancel VALUES (1,103,SYSDATE);
```

6. Seat_alloted:

```
CREATE TABLE seat_alloted (
   id NUMBER PRIMARY KEY,
   flight_no VARCHAR2(10),
   passenger_id NUMBER,
   class VARCHAR2(20),
   total_seats NUMBER,
   total_price NUMBER,
   FOREIGN KEY (flight_no) REFERENCES flight_details(flight_no),
   FOREIGN KEY (passenger_id) REFERENCES passenger_details(id_pass)
);
```

```
2   select * from seat_alloted;
```

Results | Explain | Describe | Saved SQL | History

ID	FLIGHT_NO	PASSENGER_ID	CLASS	TOTAL_SEATS	TOTAL_PRICE
1	AI101	101	Economy	2	8000
2	AI102	102	Business	1	6000

2 rows returned in 0.03 seconds Download

7. Airport:

```
CREATE TABLE airport (
   airport_code VARCHAR2(5) PRIMARY KEY,
   name VARCHAR2(100),
   city VARCHAR2(50),
   country VARCHAR2(50)
);
```

```
INSERT INTO airport VALUES ('DEL','Indira Gandhi Airport','Delhi','India');
INSERT INTO airport VALUES ('BOM','Mumbai Airport','Mumbai','India');
INSERT INTO airport VALUES ('BLR','Bangalore Airport','Bangalore','India');
```

8. Vip_Passengers:

```
CREATE TABLE vip_passengers (
   vip_id NUMBER PRIMARY KEY,
   flight_no VARCHAR2(10),
   passenger_id NUMBER,
   status VARCHAR2(20),
   privileges VARCHAR2(200),
   FOREIGN KEY (flight_no) REFERENCES flight_details(flight_no),
   FOREIGN KEY (passenger_id) REFERENCES passenger_details(id_pass)
);
```

```
2   select * from vip_passengers;
```

Results | Explain | Describe | Saved SQL | History

VIP_ID	FLIGHT_NO	PASSENGER_ID	STATUS	PRIVILEGES
1	AI101	101	Gold	Lounge access
2	AI102	102	Platinum	Priority boarding

2 rows returned in 0.02 seconds Download

9. Luggage:

```
CREATE TABLE luggage (
   luggage_id NUMBER PRIMARY KEY,
   passenger_id NUMBER,
   weight_kg NUMBER,
   description VARCHAR2(200),
   FOREIGN KEY (passenger_id) REFERENCES passenger_details(id_pass)
);

INSERT INTO luggage VALUES (1,101,10,'Backpack');
INSERT INTO luggage VALUES (2,102,15,'Suitcase');
```

10. Payment_method:

```
CREATE TABLE payment_methods (
   payment_method_id NUMBER PRIMARY KEY,
   passenger_id NUMBER,
   method_type VARCHAR2(50),
   card_number VARCHAR2(30),
   expiration_date DATE,
   billing_address VARCHAR2(200),
   FOREIGN KEY (passenger_id) REFERENCES passenger_details(id_pass)
);
```

```
2  select * from  payment_methods ;
```

Results | Explain | Describe | Saved SQL | History

PAYMENT_METHOD_ID	PASSENGER_ID	METHOD_TYPE	CARD_NUMBER	EXPIRATION_DATE	BILLING_ADDRESS
2	102	Debit Card	9876543210987654	6/1/2025	Mumbai
1	101	Credit Card	1234567890123456	12/1/2026	Delhi

2 rows returned in 0.03 seconds Download

11. Passengers_preferences:

```
CREATE TABLE passenger_preferences (
   preference_id NUMBER PRIMARY KEY,
   passenger_id NUMBER,
   seat_preference VARCHAR2(20),
   meal_preference VARCHAR2(50),
   special_assistance_needed VARCHAR2(100),
   FOREIGN KEY (passenger_id) REFERENCES passenger_details(id_pass)
);

INSERT INTO passenger_preferences VALUES (1,101,'Window','Veg','None');
INSERT INTO passenger_preferences VALUES (2,102,'Aisle','Non-Veg','Wheelchair');
```

12. Passengers_feedback:

```
CREATE TABLE passenger_feedback (
   feedback_id NUMBER PRIMARY KEY,
   passenger_id NUMBER,
   feedback VARCHAR2(200),
   feedback_date DATE,
   FOREIGN KEY (passenger_id) REFERENCES passenger_details(id_pass)
);
```

```
1
2  select * from passenger_feedback ;
```

Results | Explain | Describe | Saved SQL | History

FEEDBACK_ID	PASSENGER_ID	FEEDBACK	FEEDBACK_DATE
1	101	Good flight	4/24/2026
2	102	Delayed but okay	4/24/2026

2 rows returned in 0.02 seconds Download

Procedures

1. To increase salary of crew details on the basis of commission.

```
CREATE OR REPLACE PROCEDURE sal_incr(
   id_no IN crew_details.crew_id%TYPE,
   par IN NUMBER
)
IS
BEGIN
   UPDATE crew_details
   SET sal = sal + (sal * par / 100)
   WHERE crew_id = id_no;

   IF SQL%ROWCOUNT = 0 THEN
      DBMS_OUTPUT.PUT_LINE('No crew found');
   ELSE
      DBMS_OUTPUT.PUT_LINE('Salary updated successfully for Crew ID: ' || id_no);
   END IF;

   COMMIT;

EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error occurred: ' || SQLERRM);
      ROLLBACK;
END;
/
```

```
Procedure created.

0.08 seconds
```

```
BEGIN
    sal_incr('C001',10);
END;
/
```

Results | Explain | Describe | Saved SQL | History

```
Salary updated successfully for Crew ID: C001

Statement processed.

0.02 seconds
```

2. To Retrieve passenger luggage details.

```
CREATE OR REPLACE PROCEDURE GetPassengerLuggage(
   passenger_id_param IN NUMBER
)
IS
   v_passenger_name passenger_details.passenger_name%TYPE;
BEGIN
   -- Get passenger name
   SELECT passenger_name
   INTO v_passenger_name
   FROM passenger_details
   WHERE id_pass = passenger_id_param;

   DBMS_OUTPUT.PUT_LINE('Passenger: ' || v_passenger_name);

   -- Loop luggage
   FOR luggage_rec IN (
      SELECT luggage_id, weight_kg, description
      FROM luggage
      WHERE passenger_id = passenger_id_param
   )
   LOOP
      DBMS_OUTPUT.PUT_LINE(
         'Luggage ID: ' || luggage_rec.luggage_id ||
         ', Weight: ' || luggage_rec.weight_kg ||
         ' kg, Description: ' || luggage_rec.description
      );
   END LOOP;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Passenger not found');
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

```
Procedure created.

0.08 seconds
```

```
1  BEGIN
2      GetPassengerLuggage(101);
3  END;
4  /
```

Results | Explain | Describe | Saved SQL | History

```
Passenger: Rahul
Luggage ID: 1, Weight: 10 kg, Description: Backpack

Statement processed.

0.15 seconds
```

Functions

1. To display total number of passengers.

```
CREATE OR REPLACE FUNCTION display_total_passengers
RETURN NUMBER
IS
   total_passengers NUMBER;
BEGIN
   SELECT COUNT(*) INTO total_passengers
   FROM passenger_details;

   RETURN total_passengers;
END;
/
```

```
Function created.

0.06 seconds
```

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Total Passengers: ' || display_total_passengers);
END;
/
```

Results | Explain | Describe | Saved SQL | History

```
Total Passengers: 3

Statement processed.

0.15 seconds
```

2. To calculate price of seats selected by the passenger.

```
CREATE OR REPLACE FUNCTION calculate_price(
   f_flight_no IN VARCHAR2,
   f_class IN VARCHAR2,
   f_total_seats IN NUMBER
) RETURN NUMBER
IS
   v_price NUMBER;
   v_available NUMBER;
BEGIN
   IF UPPER(f_class) = 'BUSINESS' THEN
      SELECT business_class, business_class_price
      INTO v_available, v_price
      FROM seat_status
      WHERE flight_no = f_flight_no;
   ELSE
      SELECT economy_class, economy_class_price
      INTO v_available, v_price
      FROM seat_status
      WHERE flight_no = f_flight_no;
   END IF;

   IF f_total_seats > v_available THEN
      RAISE_APPLICATION_ERROR(-20001, 'Seats not available');
   END IF;

   RETURN v_price * f_total_seats;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20002, 'Flight not found');
   WHEN OTHERS THEN
      RAISE_APPLICATION_ERROR(-20003, SQLERRM);
END;
/
```

```
Function created.

0.06 seconds
```

```
BEGIN
    DBMS_OUTPUT.PUT_LINE(calculate_price('AI101','BUSINESS',2));
END;
/
```

Results | Explain | Describe | Saved SQL | History

```
20000

Statement processed.

0.02 seconds
```

3. To segregate vip privileges on basis of gold, silver and platinum.

```
CREATE OR REPLACE FUNCTION SegregatePrivileges(
   status_param IN VARCHAR2
) RETURN VARCHAR2
IS
BEGIN
   CASE UPPER(status_param)
      WHEN 'GOLD' THEN
         RETURN 'Priority boarding, Lounge access';
      WHEN 'SILVER' THEN
         RETURN 'Extra baggage allowance';
      WHEN 'PLATINUM' THEN
         RETURN 'Upgrade vouchers, Premium lounge access';
      ELSE
         RETURN 'No privileges defined';
   END CASE;
END;
/
```

```
Function created.

0.06 seconds
```

```
BEGIN
    DBMS_OUTPUT.PUT_LINE(SegregatePrivileges('Gold'));
END;
/
```

Results | Explain | Describe | Saved SQL | History

```
Priority boarding, Lounge access

Statement processed.
```

4. To calculate total number of years crew member has worked.

```
CREATE OR REPLACE FUNCTION CalculateYearsWorked(
   hiredate_crew IN DATE
) RETURN NUMBER
IS
BEGIN
   RETURN TRUNC(MONTHS_BETWEEN(SYSDATE, hiredate_crew) / 12);
END;
/
```

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Years Worked: ' || CalculateYearsWorked(DATE '2020-05-11'));
END;
/
```

Results | Explain | Describe | Saved SQL | History

```
Years Worked: 5

Statement processed.
```

Cursors

1. To fetch flight details from table flight_details.

```
DECLARE
   CURSOR fetch1(p_origin VARCHAR2, p_dest VARCHAR2, p_date DATE) IS
      SELECT flight_no, flight_name, origin, destination,
            departure_date, departure_time, arrival_time,
            total_distance, flight_duration, status
      FROM flight_details
      WHERE origin = p_origin
        AND destination = p_dest
        AND TRUNC(departure_date) = TRUNC(p_date);
BEGIN
   DBMS_OUTPUT.PUT_LINE('--- Flight Details ---');

   FOR x IN fetch1('Delhi','Mumbai',DATE '2024-04-24')
   LOOP
      DBMS_OUTPUT.PUT_LINE(
         x.flight_no || ' | ' ||
         x.flight_name || ' | ' ||
         x.origin || ' -> ' || x.destination || ' | ' ||
         x.departure_date || ' | ' ||
         x.status
      );
   END LOOP;
END;
/
```

```
--- Flight Details ---
AI101 | Air India | Delhi -> Mumbai | 4/24/2024 | On Time

Statement processed.
```

2. To fetch passenger details from table passenger_details.

```
DECLARE
   CURSOR fetch2(p_flight VARCHAR2, p_date DATE) IS
      SELECT flight_no, passenger_name, email, phone, gender, booking_date
      FROM passenger_details
      WHERE flight_no = p_flight
        AND TRUNC(booking_date) = TRUNC(p_date);

BEGIN
   DBMS_OUTPUT.PUT_LINE('--- Passenger Details ---');

   FOR x IN fetch2('AI101', SYSDATE)
   LOOP
      DBMS_OUTPUT.PUT_LINE(
         x.passenger_name || ' | ' ||
         x.email || ' | ' ||
         x.phone || ' | ' ||
         x.gender
      );
   END LOOP;
END;
/
```

```
--- Passenger Details ---
Rahul | rahul@gmail.com | 9876543210 | Male

Statement processed.
```

3. To calculate total number of vegetarian passengers on flight.

```
DECLARE
   v_count NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_count
   FROM passenger_preferences
   WHERE UPPER(meal_preference) = 'VEG';

   DBMS_OUTPUT.PUT_LINE('Vegetarian Count: ' || v_count);
END;
/
```

or

```
DECLARE
   v_veg_count NUMBER := 0;

   CURSOR c_passenger_preferences IS
      SELECT passenger_id
      FROM passenger_preferences
      WHERE UPPER(meal_preference) = 'VEG';

BEGIN
   FOR pref_rec IN c_passenger_preferences
   LOOP
      v_veg_count := v_veg_count + 1;
   END LOOP;

   DBMS_OUTPUT.PUT_LINE(
      'Vegetarian Passengers Count: ' || v_veg_count
   );
END;
/
```

```
Vegetarian Passengers Count: 1

Statement processed.
```

Triggers

1. To insert details of passengers who cancel their bookings into table details_cancel.

```
CREATE SEQUENCE details_cancel_seq
START WITH 2
INCREMENT BY 1;

CREATE OR REPLACE TRIGGER trig_details
AFTER DELETE ON passenger_details
FOR EACH ROW
BEGIN
   INSERT INTO details_cancel (
      cancel_id,
      passenger_id,
      cancel_date
   )
   VALUES (
      details_cancel_seq.NEXTVAL,
      :OLD.id_pass,
      SYSDATE
   );
END;
/
```

```
Trigger created.

0.06 seconds
```

```
DELETE FROM passenger_details WHERE id_pass = 101;
```

```
1   SELECT * FROM details_cancel;
```

Results | Explain | Describe | Saved SQL | History

CANCEL_ID	PASSENGER_ID	PASSENGER_NAME	CANCEL_DATE
4	101	Rahul	4/24/2026

1 rows returned in 0.02 seconds Download

2. To raise an error if luggage weight exceeds 20 kg.

```
CREATE OR REPLACE TRIGGER CheckLuggageWeight
BEFORE INSERT OR UPDATE ON luggage
FOR EACH ROW
BEGIN
   IF :NEW.weight_kg > 20 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Luggage cannot exceed 20 kg');
   END IF;
END;
/
```

```
Trigger created.

0.06 seconds
```

```
1   INSERT INTO luggage VALUES (10,101,25,'Heavy Bag');
```

Results | Explain | Describe | Saved SQL | History

```
ORA-20001: Luggage cannot exceed 20 kg
ORA-06512: at "WKSP_THAPAR.CHECKLUGGAGEWEIGHT", line 3
ORA-04088: error during execution of trigger 'WKSP_THAPAR.CHECKLUGGAGEWEIGHT'


1. INSERT INTO luggage VALUES (10,101,25,'Heavy Bag');
```

3. To raise an error if country entered in table airport is not India.

```
CREATE OR REPLACE TRIGGER CheckAirportCountry
BEFORE INSERT OR UPDATE ON airport
FOR EACH ROW
BEGIN
   IF UPPER(:NEW.country) <> 'INDIA' THEN
      RAISE_APPLICATION_ERROR(-20002, 'Only Indian airports allowed');
   END IF;
END;
/
```

```
Trigger created.

0.06 seconds
```

```
1   INSERT INTO airport VALUES ('USA','NY Airport','New York','USA');
```

Results | Explain | Describe | Saved SQL | History

```
ORA-20002: Only Indian airports allowed
ORA-06512: at "WKSP_THAPAR.CHECKAIRPORTCOUNTRY", line 3
ORA-04088: error during execution of trigger 'WKSP_THAPAR.CHECKAIRPORTCOUNTRY'


1. INSERT INTO airport VALUES ('USA','NY Airport','New York','USA');
```

SQL Queries

1.SQL Query to select details of crew whose gender is male:

SELECT crew_id, crew_name, gender, job, hiredate, sal, comm
FROM crew_details
WHERE UPPER(gender) = 'MALE';

```
SELECT crew_id, crew_name, gender, job, hiredate, sal, comm
FROM crew_details
WHERE UPPER(gender) = 'MALE';
```

Results | Explain | Describe | Saved SQL | History

CREW_ID	CREW_NAME	GENDER	JOB	HIREDATE	SAL	COMM
C001	John	Male	Pilot	5/11/2020	55000	10000
C003	Gaurav	Male	Ticket Agent	1/1/2022	20000	5000

2 rows returned in 0.02 seconds Download

2.SQL Query to delete record from vip_passengers where flight_no is AI102.

DELETE FROM vip_passengers
WHERE flight_no = 'AI102';

```
DELETE FROM vip_passengers
WHERE flight_no = 'AI102';
```

Results | Explain | Describe | Saved SQL | History

1 row(s) deleted.

0.01 seconds

3.To update the price of business class seats.

UPDATE seat_status
SET business_class_price = 15000
WHERE flight_no = 'AI101';

```
1   UPDATE seat_status
2   SET business_class_price = 15000
3   WHERE flight_no = 'AI101';
```

Results | Explain | Describe | Saved SQL | History

1 row(s) updated.

0.01 seconds

4.To display flight name and total seats from table flight_details and seat_status.

SELECT fd.flight_name, s.total_seats
FROM flight_details fd
JOIN seat_status s
ON fd.flight_no = s.flight_no
WHERE fd.flight_no = 'AI101';

```
1   SELECT fd.flight_name, s.total_seats
2   FROM flight_details fd
3   JOIN seat_status s
4   ON fd.flight_no = s.flight_no
5   WHERE fd.flight_no = 'AI101';
```

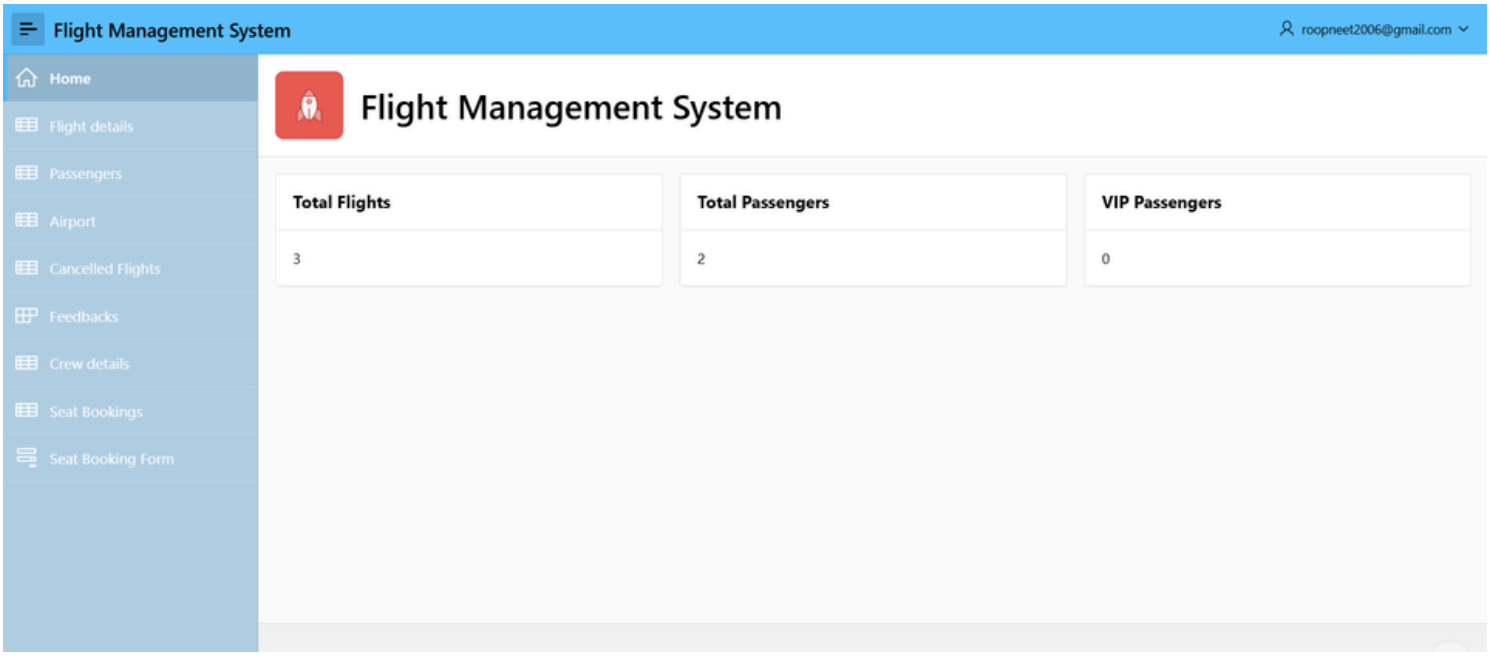
Results | Explain | Describe | Saved SQL | History

FLIGHT_NAME	TOTAL_SEATS
Air India	80

1 rows returned in 0.02 seconds Download

Frontend

UI IS MADE USING ORACLE APEX



Flight details

Flight Name	Origin	Destination	Departure Date	Departure Time	Arrival Time	Total Distance	Flight Duration	Status
Air India	Delhi	Mumbai	4/24/2024	4/24/2026	4/24/2026	1400	2.5	On Time
Indigo	Amritsar	Delhi	4/25/2024	4/24/2026	4/24/2026	200	1	On Time
SpiceJet	Bangalore	Kolkata	4/26/2024	4/24/2026	4/24/2026	1500	2.8	Delayed

Crew details

Crew Name	Gender	Job	Hiredate	Sal	Comm
John	Male	Pilot	5/11/2020	55000	10000
Mary	Female	Co-Pilot	12/10/2021	30000	5000
Gaurav	Male	Ticket Agent	1/1/2022	20000	5000

Conclusion

In conclusion, the Flight Management System developed in PL/SQL stands as a testament to the fusion of advanced technology and meticulous engineering in modern aviation. Through its robust functionality, encompassing precise navigation, efficient flight planning, and seamless integration, it exemplifies the forefront of aviation innovation. By implementing a well-structured database, users can easily access and update data, facilitating efficient decision-making processes. Furthermore, the system offers a user friendly interface and various querying capabilities, enhancing the overall user experience. Overall, this project demonstrates the importance of utilizing database management systems in the transport industry to streamline operations and improve data accuracy.